

PORTADA

Técnicas de Programación

Unidad 4 — Archivos, Errores y Pruebas: Manejo de Excepciones en Java

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

Un programa que no maneja sus errores no falla con elegancia: se detiene en seco, deja datos a medias y no le dice a nadie por qué.



TRANSICIÓN

Empieza la Unidad 4

Cerraste la Unidad 3 documentando y optimizando código correcto. Pero ningún programa real es siempre correcto: un archivo no existe, una división es por cero, un índice se sale del arreglo. La Unidad 4 empieza justo ahí — cómo **anticipar, capturar y responder** a esas fallas sin que el programa completo se venga abajo. Hoy: excepciones. La próxima sesión: archivos de texto y CSV, donde este mecanismo se vuelve indispensable.

CONCEPTO

¿Qué es una excepción?

Una **excepción** es un objeto que Java crea automáticamente cuando ocurre un evento que interrumpe el flujo normal de ejecución de un programa (una condición anómala: datos inválidos, un recurso no disponible, una operación imposible). En vez de dejar que el programa continúe con un resultado incorrecto, Java **lanza** ese objeto y detiene la ejecución normal, buscando quién esté preparado para responder a ese tipo de error.

Sin manejo de excepciones, el programa se detiene por completo (se "cae") y muestra un *stack trace* en consola. Con manejo de excepciones, el programador decide qué hacer: reintentar, mostrar un mensaje claro, registrar el error, o cerrar recursos abiertos antes de terminar.

CONCEPTO

La jerarquía: todo excepción es un `Throwable`

En Java, todo lo que se puede lanzar con `throw` desciende de la clase `Throwable`, que se divide en dos ramas:

Rama	Significa	¿Se debería capturar?
<code>Error</code>	Falla grave del entorno (memoria agotada, JVM corrupta)	No — el programa normalmente no puede recuperarse
<code>Exception</code>	Condición anómala pero manejable por el programa	Sí — es la que nos interesa como desarrolladores

Dentro de `Exception` hay otra división importante: `RuntimeException` (y sus subclases) por un lado, y el resto de `Exception` por otro — la próxima diapositiva explica por qué esa división importa tanto.

CONCEPTO

Checked vs. unchecked: dos formas de exigir manejo

	Checked	Unchecked (runtime)
Clase base	Exception (menos RuntimeException)	RuntimeException
¿El compilador obliga a manejarla?	Sí — con try/catch o declarando throws	No — el programa compila igual aunque no se maneje
¿Cuándo suele ocurrir?	Fallas externas previsibles (archivo, red, base de datos)	Errores de lógica del programador
Ejemplos	IOException , SQLException	NullPointerException , ArithmeticException , ArrayIndexOutOfBoundsException

Las *checked* representan riesgos que el propio Java sabe que existen (por ejemplo, un archivo puede no existir) y por eso obliga a decidir qué hacer con ellas **antes de compilar**. Las *unchecked* casi siempre son bugs — el compilador no las exige porque, en teoría, un código bien escrito no debería producirlas.

CONCEPTO

Sintaxis básica: try-catch-finally

El bloque `try` contiene el código que puede fallar; `catch` captura un tipo específico de excepción y decide qué hacer; `finally` se ejecuta **siempre**, ocurra o no una excepción — típicamente para liberar recursos.

```
try {  
    int[] notas = {90, 85, 78};  
    System.out.println(notas[5]);    // fuera de rango  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Índice inválido: " + e.getMessage());  
} finally {  
    System.out.println("Este bloque siempre se ejecuta.");  
}
```

CONCEPTO

Multi-catch: un bloque, varios tipos de error

Cuando dos o más tipos de excepción deben manejarse de la misma forma, se pueden capturar en un solo bloque `catch` separándolos con `|`, en vez de repetir el mismo código dos veces.

```
try {  
    conectarBaseDeDatos();  
    leerArchivoConfiguracion();  
} catch (IOException | SQLException e) {  
    System.out.println("Fallo de recurso externo: " + e.getMessage());  
}
```

Java exige que los tipos combinados con `|` no tengan relación de herencia directa entre sí (no tendría sentido capturar una clase y su propia superclase en el mismo `|`).

CONCEPTO

try-with-resources : cerrar sin olvidarlo

Cualquier clase que implemente la interfaz `AutoCloseable` (por ejemplo, las clases para leer y escribir archivos que verás la próxima sesión) puede declararse dentro del paréntesis del `try`. Java se encarga de llamar automáticamente a su método `close()` al salir del bloque, **incluso si ocurrió una excepción** — sin necesitar un `finally` manual.

```
try (Scanner lector = new Scanner(new File("datos.txt"))) {  
    while (lector.hasNextLine()) {  
        System.out.println(lector.nextLine());  
    }  
} catch (FileNotFoundException e) {  
    System.out.println("No se encontró el archivo: " + e.getMessage());  
}  
// "lector" se cierra automáticamente al terminar el try, sin código extra
```

Este patrón se vuelve central en la próxima sesión, con lectura y escritura de archivos de texto y CSV.

CONCEPTO

throw vs. throws : no son lo mismo

	throw	throws
Qué hace	Lanza una excepción, en ese punto exacto del código	Declara, en la firma del método, que ese método <i>puede</i> lanzar una excepción
Dónde va	Dentro del cuerpo de un método	En la firma del método, después de los parámetros
Cantidad	Va seguido de un solo objeto excepción	Puede listar varios tipos separados por coma

```
public void retirar(double monto) throws SaldoInsuficienteException {
    if (monto > saldo) {
        throw new SaldoInsuficienteException("Saldo insuficiente para retirar " + monto);
    }
    saldo -= monto;
}
```

CONCEPTO

Excepciones personalizadas

Cuando ninguna excepción estándar de Java describe bien un error propio del negocio (por ejemplo, "saldo insuficiente" o "cupo agotado"), conviene crear una **clase de excepción propia**, extendiendo `Exception` (checked) o `RuntimeException` (unchecked) según si se quiere obligar a manejarla.

```
public class SaldoInsuficienteException extends Exception {  
    public SaldoInsuficienteException(String mensaje) {  
        super(mensaje);    // delega el mensaje a la clase padre (Exception)  
    }  
}
```

El constructor llama a `super(mensaje)` para que ese texto quede disponible después con `e.getMessage()`, igual que en cualquier excepción de la biblioteca estándar.

CONCEPTO

¿Cuándo conviene crear una excepción propia?

- Cuando el error representa una **regla de negocio**, no un error técnico genérico (un `SaldoInsuficienteException` comunica más que un simple `RuntimeException`).
- Cuando quien llama al método necesita **distinguir ese error de otros** para reaccionar distinto (reintentar vs. cancelar la operación).
- Cuando quieres adjuntar información adicional al error (por ejemplo, el monto exacto que faltaba) a través de campos propios de la clase.

```
Cuenta cuenta = new Cuenta(50000);
try {
    cuenta.retirar(200000);
} catch (SaldoInsuficienteException e) {
    System.out.println("No se pudo procesar el retiro: " + e.getMessage());
}
```

CONTRAEJEMPLO

Excepciones "tragadas": el error que desaparece en silencio

```
try {  
    procesarPago(monto);  
} catch (Exception e) {  
    // no hacer nada  
}  
System.out.println("Pago procesado."); // ¡mentira! puede que haya fallado
```

Este patrón tiene dos problemas graves a la vez: capturar `Exception` de forma genérica (sin saber qué error específico se espera) y dejar el bloque `catch` vacío, que hace desaparecer el error sin dejar ningún rastro. El programa sigue como si nada hubiera pasado, y el mensaje final ("Pago procesado") queda siendo falso.

CONCEPTO

Buenas prácticas al manejar excepciones

- **Captura el tipo más específico posible**, no `Exception` genérica — así sabes exactamente qué falló y no ocultas errores inesperados de otro tipo.
- **Nunca dejes un `catch` vacío**. Como mínimo, registra el error (`e.getMessage()` o `e.printStackTrace()`) aunque decidas continuar la ejecución.
- **Escribe mensajes claros y accionables**: no "Error", sino algo como "No se pudo leer el archivo config.txt: verifica que exista en la ruta indicada".
- **No uses excepciones para controlar flujo normal** (por ejemplo, para salir de un ciclo) — son costosas y confunden la intención del código.

CONCEPTO

Documentar excepciones con Javadoc: @throws

Ya viste en la Unidad 3 que Javadoc documenta parámetros y retorno con `@param` y `@return` . Cuando un método puede lanzar una excepción, se documenta con `@throws` , explicando **bajo qué condición** ocurre — información que quien usa el método necesita sin tener que leer su implementación.

```
/**
 * Retira un monto de la cuenta.
 *
 * @param monto valor a retirar, debe ser mayor a 0
 * @throws SaldoInsuficienteException si el monto supera el saldo disponible
 */
public void retirar(double monto) throws SaldoInsuficienteException { ... }
```

INTERACCIÓN

Identifiquen el error y arréglenlo

En parejas: este fragmento compila pero tiene dos problemas de manejo de excepciones. Identifíquenlos y reescríbanlo.

```
public void leerNota(int[] notas, int indice) {  
    try {  
        System.out.println(notas[indice]);  
    } catch (Exception e) {  
    }  
}
```

6 minutos · pistas: ¿qué tipo específico de excepción aplica aquí? ¿qué falta dentro del catch?

SÍNTESIS

Lo que hay que llevarse de hoy

1. Una excepción interrumpe el flujo normal; sin manejarla, el programa se detiene por completo.
2. `Throwable` se divide en `Error` (no manejable) y `Exception`; dentro de `Exception`, las *checked* exigen manejo explícito y las *unchecked* (`RuntimeException`) no.
3. `try-catch-finally`, `multi-catch` y `try-with-resources` son las herramientas para capturar y limpiar recursos; `throw` lanza, `throws` declara.
4. Crea excepciones propias cuando representen reglas de negocio; nunca captures `Exception` genérica con un `catch` vacío.



CIERRE

Antes de la próxima sesión

- Practicar `try-catch-finally` y multi-catch con al menos dos tipos distintos de excepción unchecked (`ArithmeticException` , `ArrayIndexOutOfBoundsException`).
- Crear una excepción personalizada propia y usarla en un método con `throw` / `throws` .

Próxima sesión: Gestión de Archivos — Texto y CSV, donde `try-with-resources` se vuelve la herramienta central para leer y escribir sin fugas de recursos.

